code/nndl/conv_layers.py

```python
import numpy as np
from nndl.layers import (
    affine_forward,
    affine_backward,
    relu_forward,
    relu_backward,
    softmax_loss,
    batchnorm_forward,
    batchnorm_backward,
)

"""
This code was originally written for CS 231n at Stanford University
(cs231n.stanford.edu).  It has been modified in various areas for use in the
ECE 239AS class at UCLA.  This includes the descriptions of what code to
implement as well as some slight potential changes in variable names to be
consistent with class nomenclature.  We thank Justin Johnson & Serena Yeung for
permission to use this code.  To see the original version, please visit
cs231n.stanford.edu.
"""


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    pad = conv_param["pad"]
    stride = conv_param["stride"]

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the forward pass of a convolutional neural network.
```

```python
    #   Store the output as 'out'.
    #   Hint: to pad the array, you can use the function np.pad.
    # ================================================================ #

    N, C, H, W = x.shape
    F, _, HH, WW = w.shape

    stride = conv_param['stride']
    pad = conv_param['pad']

    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    out = np.zeros((N, F, H_out, W_out))

    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    for f in range(F):
        for i in range(H_out):
            for j in range(W_out):
                h_start = i * stride
                h_end = h_start + HH
                w_start = j * stride
                w_end = w_start + WW

                x_window = x_pad[:, :, h_start:h_end, w_start:w_end]
                out[:, f, i, j] = np.sum(x_window * w[f], axis=(1, 2, 3)) + b[f]

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    N, F, out_height, out_width = dout.shape
    x, w, b, conv_param = cache

    stride, pad = [conv_param["stride"], conv_param["pad"]]
    xpad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode="constant")
```

```python
    num_filts, _, f_height, f_width = w.shape

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the backward pass of a convolutional neural network.
    #   Calculate the gradients: dx, dw, and db.
    # ================================================================ #

    dxpad = np.zeros_like(xpad)
    dw = np.zeros_like(w)
    db = np.zeros_like(b)

    for n in range(N):
        for f in range(num_filts):
            for i in range(out_height):
                for j in range(out_width):
                    h_start = i * stride
                    h_end = h_start + f_height
                    w_start = j * stride
                    w_end = w_start + f_width

                    dout_val = dout[n, f, i, j]

                    db[f] += dout_val
                    dw[f] += xpad[n, :, h_start:h_end, w_start:w_end] * dout_val
                    dxpad[n, :, h_start:h_end, w_start:w_end] += w[f] * dout_val

    if pad > 0:
        dx = dxpad[:, :, pad:-pad, pad:-pad]
    else:
        dx = dxpad

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
```

```python
    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the max pooling forward pass.
    # ================================================================ #

    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']

    H_out = 1 + (H - pool_height) // stride
    W_out = 1 + (W - pool_width) // stride

    out = np.zeros((N, C, H_out, W_out))

    for i in range(H_out):
        for j in range(W_out):
            h_start = i * stride
            h_end = h_start + pool_height
            w_start = j * stride
            w_end = w_start + pool_width

            x_window = x[:, :, h_start:h_end, w_start:w_end]
            out[:, :, i, j] = np.max(x_window, axis=(2, 3))

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    x, pool_param = cache
    pool_height, pool_width, stride = (
        pool_param["pool_height"],
        pool_param["pool_width"],
        pool_param["stride"],
    )

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the max pooling backward pass.
    # ================================================================ #
```

```python

    dx = np.zeros_like(x)
    N, C, H_out, W_out = dout.shape

    for i in range(H_out):
        for j in range(W_out):
            h_start = i * stride
            h_end = h_start + pool_height
            w_start = j * stride
            w_end = w_start + pool_width

            x_window = x[:, :, h_start:h_end, w_start:w_end]

            max_val = np.max(x_window, axis=(2, 3), keepdims=True)
            mask = (x_window == max_val)

            dout_val = dout[:, :, i, j][:, :, None, None]

            dx[:, :, h_start:h_end, w_start:w_end] += mask * dout_val

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the spatial batchnorm forward pass.
    #
    #   You may find it useful to use the batchnorm forward pass you
```

```
    #   implemented in HW #4.
    # ================================================================ #

    N, C, H, W = x.shape

    x_reshaped = x.transpose(0, 2, 3, 1).reshape(N * H * W, C)

    out_reshaped, cache = batchnorm_forward(x_reshaped, gamma, beta, bn_param)

    out = out_reshaped.reshape(N, H, W, C).transpose(0, 3, 1, 2)

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """

    # ================================================================ #
    # YOUR CODE HERE:
    #   Implement the spatial batchnorm backward pass.
    #
    #   You may find it useful to use the batchnorm forward pass you
    #   implemented in HW #4.
    # ================================================================ #

    N, C, H, W = dout.shape

    dout_reshaped = dout.transpose(0, 2, 3, 1).reshape(N * H * W, C)

    dx_reshaped, dgamma, dbeta = batchnorm_backward(dout_reshaped, cache)

    dx = dx_reshaped.reshape(N, H, W, C).transpose(0, 3, 1, 2)

    # ================================================================ #
    # END YOUR CODE HERE
    # ================================================================ #

    return dx, dgamma, dbeta

```